```
from google.colab import drive
drive.mount('/content/drive')

```

```
Mounted at /content/drive
```

```
# 2. Add the directory containing 'models.py' to Python's path
import sys
sys.path.append('/content/drive/MyDrive/') # Assuming 'models.py' is directly in MyDrive

# 3. Import the module
import models
from models import *
```

```
import argparse
import os
import time
import shutil

import torch
import torch.nn as nn
import torch.optim as optim
import torch.nn.functional as F
import torch.backends.cudnn as cudnn


import torchvision
import torchvision.transforms as transforms

from models import *


global best_prec
use_gpu = torch.cuda.is_available()
print('=> Building model...')



batch_size = 128
model_name = "VGG16_quant_16x16_128"
model = VGG16_quant_16x16()

print(model)

normalize = transforms.Normalize(mean=[0.491, 0.482, 0.447], std=[0.247, 0.243, 0.262])


train_dataset = torchvision.datasets.CIFAR10(
    root='./data',
    train=True,
    download=True,
    transform=transforms.Compose([
        transforms.RandomCrop(32, padding=4),
        transforms.RandomHorizontalFlip(),
        transforms.ToTensor(),
        normalize,
    ]))
trainloader = torch.utils.data.DataLoader(train_dataset, batch_size=batch_size, shuffle=True, num_workers=2)


test_dataset = torchvision.datasets.CIFAR10(
    root='./data',
    train=False,
    download=True,
    transform=transforms.Compose([
        transforms.ToTensor(),
        normalize,
    ]))

testloader = torch.utils.data.DataLoader(test_dataset, batch_size=batch_size, shuffle=False, num_workers=2)


print_freq = 100 # every 100 batches, accuracy printed. Here, each batch includes "batch_size" data points
# CIFAR10 has 50,000 training data, and 10,000 validation data.

def train(trainloader, model, criterion, optimizer, epoch):
    batch_time = AverageMeter()
    data_time = AverageMeter()
    losses = AverageMeter()
    top1 = AverageMeter()
```

```
    top1 = AverageMeter()

    model.train()

    end = time.time()
    for i, (input, target) in enumerate(trainloader):
        # measure data loading time
        data_time.update(time.time() - end)

        input, target = input.cuda(), target.cuda()

        # compute output
        output = model(input)
        loss = criterion(output, target)

        # measure accuracy and record loss
        prec = accuracy(output, target)[0]
        losses.update(loss.item(), input.size(0))
        top1.update(prec.item(), input.size(0))

        # compute gradient and do SGD step
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        # measure elapsed time
        batch_time.update(time.time() - end)
        end = time.time()


        if i % print_freq == 0:
            print('Epoch: [{0}][{1}/{2}]\t'
                  'Time {batch_time.val:.3f} ({batch_time.avg:.3f})\t'
                  'Data {data_time.val:.3f} ({data_time.avg:.3f})\t'
                  'Loss {loss.val:.4f} ({loss.avg:.4f})\t'
                  'Prec {top1.val:.3f}% ({top1.avg:.3f}%)'.format(
                   epoch, i, len(trainloader), batch_time=batch_time,
                   data_time=data_time, loss=losses, top1=top1))



def validate(val_loader, model, criterion ):
    batch_time = AverageMeter()
    losses = AverageMeter()
    top1 = AverageMeter()

    # switch to evaluate mode
    model.eval()

    end = time.time()
    with torch.no_grad():
        for i, (input, target) in enumerate(val_loader):

            input, target = input.cuda(), target.cuda()

            # compute output
            output = model(input)
            loss = criterion(output, target)

            # measure accuracy and record loss
            prec = accuracy(output, target)[0]
            losses.update(loss.item(), input.size(0))
            top1.update(prec.item(), input.size(0))

            # measure elapsed time
            batch_time.update(time.time() - end)
            end = time.time()

            if i % print_freq == 0:  # This line shows how frequently print out the status. e.g., i%5 => every 5 batch,
                print('Test: [{0}/{1}]\t'
                  'Time {batch_time.val:.3f} ({batch_time.avg:.3f})\t'
                  'Loss {loss.val:.4f} ({loss.avg:.4f})\t'
                  'Prec {top1.val:.3f}% ({top1.avg:.3f}%)'.format(
                   i, len(val_loader), batch_time=batch_time, loss=losses,
                   top1=top1))

    print(' * Prec {top1.avg:.3f}% '.format(top1=top1))
    return top1.avg


def accuracy(output, target, topk=(1,)):
    """Computes the precision@k for the specified values of k"""
    maxk = max(topk)
```

```
    batch_size = target.size(0)

    _, pred = output.topk(maxk, 1, True, True)
    pred = pred.t()
    correct = pred.eq(target.view(1, -1).expand_as(pred))

    res = []
    for k in topk:
        correct_k = correct[:k].view(-1).float().sum(0)
        res.append(correct_k.mul_(100.0 / batch_size))
    return res


class AverageMeter(object):
    """Computes and stores the average and current value"""
    def __init__(self):
        self.reset()

    def reset(self):
        self.val = 0
        self.avg = 0
        self.sum = 0
        self.count = 0

    def update(self, val, n=1):
        self.val = val
        self.sum += val * n
        self.count += n
        self.avg = self.sum / self.count


def save_checkpoint(state, is_best, fdir):
    filepath = os.path.join(fdir, 'checkpoint.pth')
    torch.save(state, filepath)
    if is_best:
        shutil.copyfile(filepath, os.path.join(fdir, 'model_best.pth.tar'))


def adjust_learning_rate(optimizer, epoch):
    """For resnet, the lr starts from 0.1, and is divided by 10 at 80 and 120 epochs"""
    adjust_list = [300,400,550]
    if epoch in adjust_list:
        for param_group in optimizer.param_groups:
            param_group['lr'] = param_group['lr'] * 0.1

#model = nn.DataParallel(model).cuda()
#all_params = checkpoint['state_dict']
#model.load_state_dict(all_params, strict=False)
#criterion = nn.CrossEntropyLoss().cuda()
#validate(testloader, model, criterion)
```

```
  )
  (15): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
```

```
    (33): QuantConv2d(
      256, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
      (weight_quant): weight_quantize_fn()
    )
    (34): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (35): ReLU(inplace=True)
    (36): QuantConv2d(
      512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
      (weight_quant): weight_quantize_fn()
    )
    (37): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (38): ReLU(inplace=True)
    (39): QuantConv2d(
      512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
      (weight_quant): weight_quantize_fn()
    )
    (40): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (41): ReLU(inplace=True)
    (42): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (43): AvgPool2d(kernel_size=1, stride=1, padding=0)
  )
  (classifier): Linear(in_features=512, out_features=10, bias=True)
)
100%|██████████| 170M/170M [00:01<00:00, 104MB/s]
```

```
1 model.features[27] ## Modified layer
```

```
QuantConv2d(
  16, 16, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
  (weight_quant): weight_quantize_fn()
)
```

```
1 torch.any(model.features[27].weight_q != 0) # all weights 0 because hasn't trained yet.
```

```
tensor(False)
```

Training:

Use the below cells to train a new Quantized VGGNet16 with its 27th feature qunatized convolutional layer hvaing inputxoutput channels equal to 16x16.

```
1 # This cell is from the website
2
3 lr = 8e-3
4
5
6
7 weight_decay = 1e-4
8 epochs = 600
9 best_prec = 0
10
11 model = model.cuda()
12 criterion = nn.CrossEntropyLoss().cuda()
13 optimizer = torch.optim.SGD(model.parameters(), lr=lr, momentum=0.9, weight_decay=weight_decay) #changed momentum and l
14 # weight decay: for regularization to prevent overfitting
```

```
1
2
3 if not os.path.exists('result'):
4     os.makedirs('result')
5
6 fdir = 'result/'+str(model_name)
7
8 if not os.path.exists(fdir):
9     os.makedirs(fdir)
10
```

```
1
2 for epoch in range(0, epochs):
3     #adjust_learning_rate(optimizer, epoch)
4
5     train(trainloader, model, criterion, optimizer, epoch)
6
7     # evaluate on test set
8     print("Validation starts")
9     prec = validate(testloader, model, criterion)
10
11     # remember best precision and save checkpoint
12     is_best = prec > best_prec
```

```
    best_prec = max(prec,best_prec)
    print('best acc: {:1f}'.format(best_prec))
    save_checkpoint({
        'epoch': epoch + 1,
        'state_dict': model.state_dict(),
        'best_prec': best_prec,
        'optimizer': optimizer.state_dict(),
    }, is_best, fdir)

    if prec > 90: break

```

```
# !cp -r  /content/result/VGG16_quant_16x16_128 /content/drive/MyDrive/result
```

Results of training Quantized VGGNet16 with 2-bit activations:

- Accuracy = 85%

```
PATH = "/content/drive/MyDrive/result/VGG16_quant_16x16_256/model_best.pth.tar"
checkpoint = torch.load(PATH)
model.load_state_dict(checkpoint['state_dict'])
device = torch.device("cuda")

model.cuda()
model.eval()

test_loss = 0
correct = 0

with torch.no_grad():
    for data, target in testloader:
        data, target = data.to(device), target.to(device) # loading to GPU
        output = model(data)
        pred = output.argmax(dim=1, keepdim=True)
        correct += pred.eq(target.view_as(pred)).sum().item()

test_loss /= len(testloader.dataset)

print('\nTest set: Accuracy: {}/{} ({:.0f}%)\n'.format(
        correct, len(testloader.dataset),
        100. * correct / len(testloader.dataset)))
```

```
Test set: Accuracy: 8451/10000 (85%)
```

```
class SaveOutput:
    def __init__(self):
        self.outputs = []
    def __call__(self, module, module_in):
        self.outputs.append(module_in)
    def clear(self):
        self.outputs = []

######### Save inputs from selected layer ##########
save_output = SaveOutput()
i = 0
count = 0
for layer in model.modules():
    i = i+1
    if isinstance(layer, QuantConv2d):
        count += 1
        print(i,"-th layer prehooked \n", layer, count) #'count' here is just for me to check the index of save_output.
        layer.register_forward_pre_hook(save_output)
#####################################################
model.cuda()
dataiter = iter(testloader)
images, labels = next(dataiter)
images = images.cuda()
out = model(images)
```

```
 64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
```

```
16 -th layer prehooked
 QuantConv2d(
  128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
  (weight_quant): weight_quantize_fn()
) 4
21 -th layer prehooked
 QuantConv2d(
  128, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
  (weight_quant): weight_quantize_fn()
) 5
25 -th layer prehooked
 QuantConv2d(
  256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
  (weight_quant): weight_quantize_fn()
) 6
29 -th layer prehooked
 QuantConv2d(
  256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
  (weight_quant): weight_quantize_fn()
) 7
34 -th layer prehooked
 QuantConv2d(
  256, 16, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
  (weight_quant): weight_quantize_fn()
) 8
38 -th layer prehooked
 QuantConv2d(
  16, 16, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
  (weight_quant): weight_quantize_fn()
) 9
41 -th layer prehooked
 QuantConv2d(
  16, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
  (weight_quant): weight_quantize_fn()
) 10
46 -th layer prehooked
 QuantConv2d(
  256, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
  (weight_quant): weight_quantize_fn()
) 11
50 -th layer prehooked
 QuantConv2d(
  512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
  (weight_quant): weight_quantize_fn()
) 12
54 -th layer prehooked
 QuantConv2d(
  512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
  (weight_quant): weight_quantize_fn()
) 13
```

```
1 weight_q = model.features[27].weight_q
2 w_alpha = model.features[27].weight_quant.wgt_alpha
3 w_bit = 4
4
5 weight_int = weight_q / (w_alpha / (2**(w_bit-1)-1))
6
```

```
1 act = save_output.outputs[8][0]
2 act_alpha  = model.features[27].act_alpha
3 act_bit = 2
4 act_quant_fn = act_quantization(act_bit)
5
6 act_q = act_quant_fn(act, act_alpha)
7
8 act_int = act_q / (act_alpha / (2**act_bit-1))
9 #print(act_int )
```

```
1 save_output.outputs[9][0].shape
```

```
torch.Size([128, 16, 4, 4])
```

```
1 conv_int = torch.nn.Conv2d(in_channels = 8, out_channels=8, kernel_size = 3, padding=1)
2 conv_int.weight = torch.nn.parameter.Parameter(weight_int)
3 relu = torch.nn.ReLU()
4 conv_int.bias = model.features[27].bias
5 output_int = conv_int(act_int)
6 output_recovered = output_int * (act_alpha / (2**act_bit-1)) * (w_alpha / (2**(w_bit-1)-1))
7 output_recovered_relu = relu(output_recovered)
```

```
1 difference = abs(save_output.outputs[9][0] - output_recovered_relu )
2 print(difference.mean())  ## It should be small, e.g.,2.3 in my trainned model
```

```
tensor(1.4224e-06, device='cuda:0', grad_fn=<MeanBackward0>)
```

```
a_int = act_int[2,:,:,:]  # pick only one input out of batch

w_int = torch.reshape(weight_int, (weight_int.size(0), weight_int.size(1), -1))  # merge ki, kj index to kij

```

```

padding = 1
stride = 1
array_size = 8 # row and column number

```

```

nig = range(a_int.size(1))  ## ni group
njg = range(a_int.size(2))  ## nj group
print(nig, njg)
```

```
range(0, 4) range(0, 4)
```

```

icg = range(int(w_int.size(1)))  ## input channel
ocg = range(int(w_int.size(0)))  ## output channel
print(icg, ocg)
```

```
range(0, 16) range(0, 16)
```

```

ic_tileg = range(int(len(icg)/(array_size*2)))
oc_tileg = range(int(len(ocg)/array_size))
print(ic_tileg, oc_tileg)
```

```
range(0, 1) range(0, 2)
```

```

kijg = range(w_int.size(2))
ki_dim = int(math.sqrt(w_int.size(2)))  ## Kernel's 1 dim size
print(kijg, ki_dim)
```

```
range(0, 9) 3
```

```

######## Padding before Convolution #######
a_pad = torch.zeros(len(icg), len(nig)+padding*2, len(nig)+padding*2).cuda()
```

```


a_pad[ :, padding:padding+len(nig), padding:padding+len(njg)] = a_int.cuda()
a_pad = torch.reshape(a_pad, (a_pad.size(0), -1))
```

```


a_tile = torch.zeros(len(ic_tileg), array_size*2,    a_pad.size(1)).cuda()
w_tile = torch.zeros(len(oc_tileg)*len(ic_tileg), array_size, array_size*2, len(kijg)).cuda() #in channel became array_

```

Only output channel is required to be tiled since each PE Computes the MAC for two input channels

```

for ic_tile in ic_tileg:
    a_tile[ic_tile,:,:] = a_pad[ic_tile*array_size*2:(ic_tile+1)*array_size*2,:]

for ic_tile in ic_tileg:
    for oc_tile in oc_tileg:
        w_tile[oc_tile,:,:,:] = w_int[oc_tile*array_size:(oc_tile+1)*array_size, :, :]
        # first dim of w_tile changed from oc_tile*len(oc_tileg) + ic_tile



```

Comparing our new tensor's which stores the 2 output channel tiles with the original tensor holding our weights. We see that w_tile correctly stores each output channel tile

```
print((w_tile[1,:,:,:]-w_int[8:16,:,:]).sum())
```

```
tensor(0., device='cuda:0', grad_fn=<SumBackward0>)
```

```
1
2 ##########################################
3
4 p_nijg = range(a_pad.size(1)) ## psum nij group
5 print(p_nijg)
```

```
range(0, 36)
```

```
1
2 psum = torch.zeros(len(ic_tileg), len(oc_tileg), array_size, len(p_nijg), len(kijg)).cuda()
3
4 for kij in kijg:
5     for ic_tile in ic_tileg:       # Tiling into array_sizeXarray_size array
6         for oc_tile in oc_tileg:   # Tiling into array_sizeXarray_size array
7             for nij in p_nijg:       # time domain, sequentially given input
8                     m = nn.Linear(2*array_size, array_size, bias=False)
9                     #m.weight = torch.nn.Parameter(w_int[oc_tile*array_size:(oc_tile+1)*array_size, ic_tile*array_size:
10                    m.weight = torch.nn.Parameter(w_tile[oc_tile,:,:,kij])
11                    psum[ic_tile, oc_tile, :, nij, kij] = m(a_tile[ic_tile,:,nij]).cuda()
12
```

```
1 import math
2
3 a_pad_ni_dim = int(math.sqrt(a_pad.size(1))) # 32
4 print(a_pad_ni_dim)
```

```
6
```

```
1
2 o_ni_dim = int((a_pad_ni_dim - (ki_dim- 1) - 1)/stride + 1)
3 o_nijg = range(o_ni_dim**2)
4 print(o_ni_dim, o_nijg)
5 print(psum.shape)
```

```
4 range(0, 16)
torch.Size([1, 2, 8, 36, 9])
```

```
1
2 out = torch.zeros(len(ocg), len(o_nijg)).cuda()
3
4
5 ### SFP accumulation ###
6 for o_nij in o_nijg:
7     for kij in kijg:
8         for ic_tile in ic_tileg:
9             for oc_tile in oc_tileg:
10                out[oc_tile*array_size:(oc_tile+1)*array_size, o_nij] = out[oc_tile*array_size:(oc_tile+1)*array_size,
11                psum[ic_tile, oc_tile, :, int(o_nij/o_ni_dim)*a_pad_ni_dim + o_nij%o_ni_dim + int(kij/ki_dim)*a_pad_ni_
12                #print(int(o_nij/o_ni_dim)*a_pad_ni_dim + o_nij%o_ni_dim + int(kij/ki_dim)*a_pad_ni_dim + kij%ki_dim)
13                ## 4th index = (int(o_nij/30)*32 + o_nij%30) + (int(kij/3)*32 + kij%3)
```

Our manual calculation of the output features before activation matches the model's saved output features.

```
1 out_2D = torch.reshape(out, (out.size(0), o_ni_dim, -1))
2 difference = (out_2D - output_int[2,:,:,:])
3 print(difference.sum())
```

```
tensor(-2.0981e-05, device='cuda:0', grad_fn=<SumBackward0>)
```

```
1 # print(out.shape)
2 # # print(relu(out))
```

```
1 # print(output_int.shape)
2 # print(out.shape)
3 # print(a_tile.shape)
4
```

The next cell produces one txt file with the input activation map. Each PE processess two subsequent input channels, and each activation is two bits. Thus each row in the MAC will receive pairs of 2 2-bit activations to process.

```
####### Store activation ####################

tile_id = 0
X = a_tile[tile_id,:,:]  # [tile_num, array row num, time_steps]

print(X.shape)
print(torch.max(X))

bit_precision = 2 #changed from 4 to 2 because activations are 2 bits each
file = open('activation_tile0.txt', 'w') #write to file
file.write('#time0row7[msb-lsb],time0row6[msb-lst],....,time0row0[msb-lst]#\n')
file.write('#time1row7[msb-lsb],time1row6[msb-lst],....,time1row0[msb-lst]#\n')
file.write('#...............#\n')

for i in range(X.size(1)):  # time step
    for j in range(X.size(0)): # row #
        if j%2==0:
          X_bin = '0000'
          for k in range(bit_precision*2):
              file.write(X_bin[k])
        X_bin = '{0:02b}'.format(round(X[15-j,i].item())) #X_bin = '{0:04b}'.format(round(X[7-j,i].item()))
        for k in range(bit_precision):
            file.write(X_bin[k])
    file.write('\n')
file.close() #close file 0000 01 10

```

```
torch.Size([16, 36])
tensor(3., device='cuda:0', grad_fn=<MaxBackward1>)
```

The next cell produces 2 sets of weight txt files one for each output tile.

```
####### store weight(kij) ###############

bit_precision = 4

for tile_id in range(2):
  for kij in range(9):
      file_name = f'weight_itile0_otile{tile_id}_kij{kij}.txt'
      W = w_tile[tile_id,:,:,kij] # w_tile[tile_num, array col num, array row num, kij]

      file = open(file_name, 'w')  #write to file
      file.write('#col0row7[msb-lsb],col0row6[msb-lst],....,col0row0[msb-lst]#\n')
      file.write('#col1row7[msb-lsb],col1row6[msb-lst],....,col1row0[msb-lst]#\n')
      file.write('#...............#\n')

      for i in range(W.size(0)):  # time step
          for j in range(W.size(1)): # row #
              if (W[i, 15-j].item() >= 0):
                  W_bin = '{0:04b}'.format(round(W[i, 15-j].item()))
              else:
                  W_bin = '{0:04b}'.format(round(W[i, 15-j].item())+16)
              for k in range(bit_precision):
                  file.write(W_bin[k])
          file.write('\n')
      file.close() #close file
```

```
# print(out.shape)
# out_int = output_int[2,:,:,:]
# out_int = out_int.reshape(out_int.size(0),-1)
# print(out_int.shape)
# (torch.round(out_int) - torch.round(out)).sum()
```

The next cell produces the output feature maps for each output tile to be used in the testbench

```
bit_precision = 16



for o_tile in range(2):
  file = open(f'out{o_tile}.txt', 'w') #write to file
  file.write('#time0col7[msb-lsb],time0col6[msb-lst],....,time0col0[msb-lst]#\n')
  file.write('#time1col7[msb-lsb],time1col6[msb-lst],....,time1col0[msb-lst]#\n')
  file.write('#...............#\n')
  out_relu = relu(out[o_tile*array_size:(o_tile+1)*array_size,:])
  for i in range(16): #o_nij
      #print(out_relu[:,i])
      for j in range(8): #out_channel
```

```
          #Out_bin = '{0:016b}'.format(round(out_relu[7-j,i].item())+65536)
          if (out_relu[7-j, i].item() >= 0):
              Out_bin = '{0:016b}'.format(round(out_relu[7-j,i].item()))
          else:
              Out_bin = '{0:016b}'.format(round(out_relu[7-j,i].item())+65536)
          for k in range(bit_precision):
              file.write(Out_bin[k])
      file.write('\n')
  file.close() #close file
```